

ARHIA ATK Reference Implementation Pack

Demonstrable proof of the security nucleus with FastAPI, OPA and JWT

Version: 2.1

Audience: Engineering leads, platform teams, CISO review teams and demo stakeholders

Classification: Confidential / Controlled use

Change log v2.0: Upgraded from demonstration script to FastAPI service. Added OPA integration, JWT capability tokens with expiration enforcement, MCP interceptor stub, workbench approval simulation, structured AIBOM validation, and comprehensive README. v2.1: OPA/Rego evaluation via subprocess with Python fallback. Real AIBOM validation per Annex K.2 seven-check sequence. All 6 ATK decision outcomes implemented (allow, allow_with_restrictions, require_human_approval, deny, degrade_autonomy, quarantine_actor). JWT audience verification enabled. Ledger entries HMAC-signed for tamper evidence. Workbench approval endpoint. Ledger integrity verification endpoint. Rego policy rewritten with mutually exclusive reason codes and correct precedence. 11 OPA test cases. Dockerfile and integration test suite. Prominent security warnings for demo mode.

1. Scope of the reference implementation

The reference implementation proves the security nucleus of ARHIA in runnable code. It demonstrates the complete decision chain: policy evaluation through OPA/Rego with automatic Python fallback when OPA is unavailable, all six ATK decision outcomes (allow, allow_with_restrictions, require_human_approval, deny, degrade_autonomy, quarantine_actor), real AIBOM validation per Annex K.2 seven-check sequence, ephemeral JWT capability-token issuance with expiration enforcement and audience claims, synchronous MCP authorization via interceptor with timing budget tracking, asynchronous evidence enrichment, immutable append-only HMAC-signed ledgering with hash-chain integrity and tamper detection, workbench approval endpoint with operator decision capture, and ledger integrity verification. The Rego policy uses mutually exclusive reason codes with correct precedence and is validated by 11 OPA test cases.

This is a reference implementation, not a production deployment. It is designed for: (a) customer-facing demonstrations during sales meetings and due diligence; (b) engineering team evaluation of the architecture's mechanics; (c) CISO review of the trust decision chain; (d) a starting point for a production implementation.

2. Components

Component	Technology	Purpose
Trust service	FastAPI (Python)	HTTP service exposing ATK endpoints: /trust/evaluate-action, /trust/revoke-token, /trust/quarantine-actor, /trust/degrade-autonomy, /trust/decision/{trace_id}, /health
Policy adapter	OPA (Open Policy Agent) via REST or embedded Rego evaluation	Evaluates the canonical ATK input against versioned Rego policy bundles. Returns allow/deny/degrade decisions.
Capability token issuer	PyJWT (RS256 or HS256)	Issues JWT tokens with: sub (actor_id), cap (capability_id), aud (target MCP server), exp (expiration), scope (zone, classification ceiling), trace_id
MCP interceptor stub	FastAPI middleware	Validates capability tokens before forwarding MCP calls. Implements synchronous auth path. Logs evidence asynchronously.
Workbench simulator	CLI-based approval flow	Simulates human approval for require_human_approval decisions. Demonstrates evidence presentation and decision capture.
Immutable ledger	JSONL with hash-chain	Append-only ledger where each entry references the previous entry's SHA-256 hash. Tampering detection built in.

AIBOM validator	JSON schema validation	Validates AIBOM records against the Annex K schema. Checks signature_status, hash, zone_allowance, approval_status and staleness.
Demo script	Python CLI	Runs the complete demonstration sequence end-to-end with colored output.

3. Demonstration sequence

The demo script executes the following sequence, printing each step with evidence:

Step 1 — Business request: A simulated payroll agent (agent-payroll-01) requests to create a batch payment. The request includes all five ATK input envelopes.

Step 2 — AIBOM validation: The trust service validates the AIBOM for the model used by the payroll agent. Signature, hash, zone allowance and approval status are checked.

Step 3 — Policy evaluation: The trust service submits the canonical JSON input to the OPA policy engine. The Rego policy evaluates zone, classification, autonomy, AIBOM trust and drift.

Step 4 — Decision and token: If the policy returns allow, the trust service issues a JWT capability token with constraints. The token, explanation record and decision are logged.

Step 5 — MCP authorization: The demo simulates an MCP call to payments.createBatch. The MCP interceptor validates the JWT token (signature, expiration, audience, scope) before authorizing the call.

Step 6 — Evidence and ledger: An immutable ledger entry is created with the decision, trace_id, token hash, MCP outcome and hash-chain reference to the previous entry.

Step 7 — Deny scenario: The demo then replays the request with zone_allowed=false, showing the deny path with reason code, explanation record and ledger entry.

Step 8 — Quarantine scenario: The demo triggers an AIBOM hash mismatch, showing the quarantine_actor path with immediate token revocation and incident record.

Step 9 — Workbench scenario: The demo triggers a require_human_approval decision, showing the workbench simulation with evidence display and approval capture.

4. Deliverables

File	Description
arhia_atk_service.py	FastAPI trust service with all ATK endpoints, JWT issuance, AIBOM validation and ledger
policy/authz.rego	Expanded OPA/Rego policy bundle with zone, classification, autonomy, AIBOM and drift rules
policy/authz_test.rego	OPA test file with 11 test cases covering decisions, reason codes, precedence and mutual exclusivity
aibom.json	Sample AIBOM file with all Annex K minimum fields
mcp_interceptor.py	MCP interceptor stub with JWT validation and evidence logging
demo.py	End-to-end demonstration script with colored output for all scenarios
ledger.jsonl	Generated immutable ledger file (created by running demo.py)
README.md	Setup instructions, prerequisites, run commands and architecture overview

5. Prerequisites and setup

Python 3.11+, pip install fastapi uvicorn pyjwt[crypto] httpx. OPA binary (optional; the reference implementation includes an embedded Rego evaluator fallback). Run: python demo.py for the full demonstration, or uvicorn arhia_atk_service:app --port 8100 for the standalone service.

6. Security notes for reviewers

SECURITY WARNING: This is a reference implementation for demonstration purposes. The source code contains a prominent security warning block at the top of arhia_atk_service.py. The following items are intentionally simplified and must be hardened for production: (a) JWT signing uses HS256 with a demo secret (env ATK_JWT_SECRET); production must use RS256 with key management via HashiCorp Vault or equivalent. (b) The ledger uses file-based JSONL with HMAC signing using a demo key (env ATK_LEDGER_HMAC_KEY); production must use a replicated database with the HMAC key stored in an HSM or cloud KMS. (c) The MCP interceptor is a stub; production must integrate with the actual MCP protocol layer. (d) OPA policies are evaluated via subprocess; production should use an OPA sidecar or OPA-as-a-service with bundle distribution. (e) No TLS is configured; production requires mutual TLS between all plane services. (f) Secrets are in environment variables; production requires a secret management service with rotation. The service emits startup warnings when running with default demo secrets.

End of ARHIA ATK Reference Implementation Pack v2.1.